



Apprentissage fédéré basé sur le clustering  
pour la Prédiction du Diabète  
Diyabet Tahmini için Küme Tabanlı Federe  
Öğrenme

Alp Özcan — 22401007

# Table des matières

|          |                                                                                        |          |
|----------|----------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Exigences, Contraintes et Conditions Réelles à Considérer pendant la Conception</b> | <b>2</b> |
| 1.1      | Économiques . . . . .                                                                  | 2        |
| 1.2      | Environnementales . . . . .                                                            | 2        |
| 1.3      | Durabilité . . . . .                                                                   | 2        |
| 1.4      | Scalabilité . . . . .                                                                  | 2        |
| 1.5      | Éthiques . . . . .                                                                     | 2        |
| 1.6      | Vie privée et sécurité . . . . .                                                       | 3        |
| 1.7      | Sociales et politiques . . . . .                                                       | 3        |
| <b>2</b> | <b>Design Conceptuel</b>                                                               | <b>3</b> |
| 2.1      | Architecture globale . . . . .                                                         | 3        |
| 2.2      | Flux de données . . . . .                                                              | 4        |
| 2.3      | Pourquoi les Processus Gaussiens? . . . . .                                            | 4        |
| 2.4      | Stratégie de clustering . . . . .                                                      | 5        |
| <b>3</b> | <b>Conception Technique</b>                                                            | <b>5</b> |
| 3.1      | Diagramme de classes UML . . . . .                                                     | 5        |
| 3.2      | Format de sérialisation des hyperparamètres . . . . .                                  | 5        |
| 3.3      | Agrégation fédérée . . . . .                                                           | 6        |
| 3.4      | Gestion de la dimensionnalité . . . . .                                                | 6        |
| <b>4</b> | <b>Analyse de Coûts</b>                                                                | <b>6</b> |
| 4.1      | Ressources matérielles . . . . .                                                       | 6        |
| 4.2      | Ressources logicielles . . . . .                                                       | 6        |
| 4.3      | Temps de calcul . . . . .                                                              | 7        |
| 4.4      | Complexité mémoire . . . . .                                                           | 7        |
| 4.5      | Taille des datasets . . . . .                                                          | 7        |
| 4.6      | Effort de développement estimé . . . . .                                               | 7        |
| <b>5</b> | <b>Premiers Résultats</b>                                                              | <b>8</b> |
| 5.1      | Dataset 1 — DPD (~100 000 patients) . . . . .                                          | 8        |
| 5.2      | Dataset 2 — Pima Indians (768 patients) . . . . .                                      | 9        |
| 5.3      | Dataset 3 — BRFSS (~229 000 patients) . . . . .                                        | 10       |
| 5.4      | Incertitude GP — Dataset 1 . . . . .                                                   | 11       |

# 1 Exigences, Contraintes et Conditions Réelles à Considérer pendant la Conception

## 1.1 Économiques

Le projet utilise uniquement des bibliothèques open-source : `PyTorch`, `GPyTorch`, `scikit-learn`, `pandas` et `matplotlib`. Le coût en licences est donc nul.

Le matériel utilisé est un GPU intégré AMD Radeon 780M, disponible sur un ordinateur personnel standard. Aucun service cloud payant n'a été nécessaire. Cette contrainte budgétaire a directement influencé certains choix techniques. Par exemple, le `SparseGP` a été intégré comme alternative à l'`ExactGP` pour les grands datasets, car l'`ExactGP` devient trop coûteux en mémoire à grande échelle.

## 1.2 Environnementales

Le GPU intégré AMD Radeon 780M consomme entre 15 et 28 W selon la charge. C'est bien moins qu'un GPU discret haute performance. De plus, le `SparseGP` réduit la complexité de  $O(n^3)$  à  $O(nm^2)$ , ce qui diminue le nombre de cycles de calcul nécessaires. Ces deux points réduisent l'empreinte énergétique du projet, même si cela n'était pas l'objectif principal.

## 1.3 Durabilité

Le code est découpé en modules indépendants : `Client.py`, `Server.py`, `clustering.py`, `evaluation.py`, etc. Cette organisation facilite la maintenance. Par ailleurs, le système normalise automatiquement le nom de la colonne cible (par exemple, `Outcome` ou `Diabetes_binary` sont tous renommés en "diabetes"). Il recalcule aussi les colonnes attendues via `expected_columns`. Ces deux mécanismes permettent d'ajouter un nouveau dataset sans modifier le reste du code.

## 1.4 Scalabilité

En théorie, le nombre de clients fédérés peut augmenter sans changer le protocole d'agrégation. En pratique, l'`ExactGP` a une complexité mémoire de  $O(n^2)$ , ce qui le rend inutilisable pour des clusters de plus de 5 000 observations sur ce matériel. Le `SparseGP` avec  $m$  points inducteurs résout ce problème. Une contrainte minimale a aussi été fixée : `MIN_CLUSTER_SIZE = 20`. Tout cluster plus petit est ignoré, pour éviter l'entraînement sur des données insuffisantes.

## 1.5 Éthiques

Un modèle de prédiction médicale peut être injuste envers certains groupes de patients, notamment les minorités ou les groupes sous-représentés. La stratégie du projet est de créer un modèle GP spécialisé par groupe (cluster), plutôt qu'un seul modèle global. L'idée est qu'un modèle entraîné sur des données homogènes est plus équitable pour ce groupe. Cela dit, certains clusters ont un déséquilibre des classes très fort (ex. cluster Normal pour `HbA1c_level` avec  $F1 = 0.000$ ). Ce problème reste un défi non résolu à ce stade.

## 1.6 Vie privée et sécurité

C'est la contrainte centrale du projet. Aucune donnée patient brute ne circule entre clients ou vers le serveur. Seuls les hyperparamètres du noyau GP sont transmis — un vecteur de petite taille contenant output scale, lengthscales et constante de moyenne. Cette approche respecte le principe de base du Federated Learning. Limite actuelle : il n'y a pas de mécanisme de chiffrement des paramètres échangés. C'est une amélioration à envisager avant tout déploiement réel.

## 1.7 Sociales et politiques

Les données de santé sont encadrées par des réglementations strictes comme le RGPD en Europe. Ces règles interdisent le transfert de données médicales non anonymisées entre institutions. Le Federated Learning répond directement à cette contrainte : les hôpitaux peuvent collaborer sans partager leurs données. Les trois datasets utilisés ici sont tous publics et anonymisés, ce qui garantit la conformité légale pendant le développement.

# 2 Design Conceptuel

## 2.1 Architecture globale

Le système suit une architecture fédérée classique : un serveur central et plusieurs clients locaux. Chaque client représente un groupe de patients (un cluster). Le serveur ne voit jamais les données — il reçoit uniquement les hyperparamètres des clients, les agrège, puis renvoie le résultat.

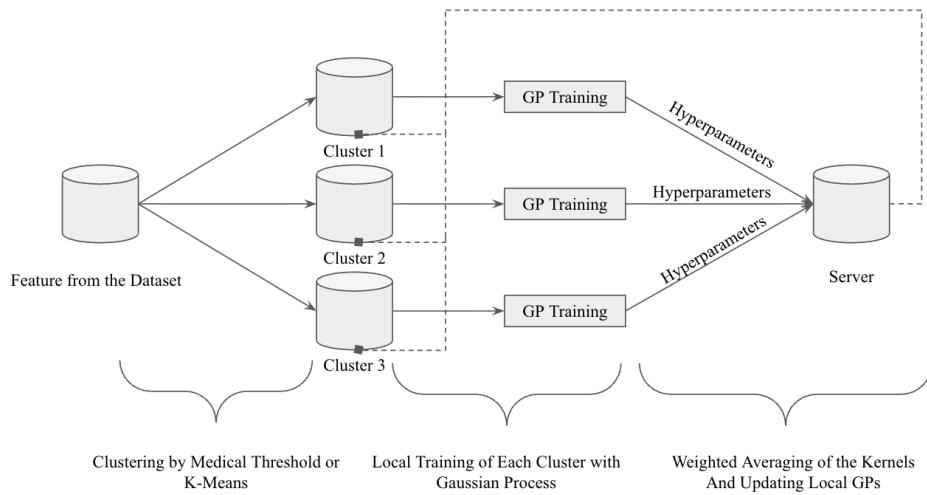


FIGURE 1 – Architecture générale du système GP-FL

## 2.2 Flux de données

Le pipeline suit cinq étapes dans l'ordre suivant :

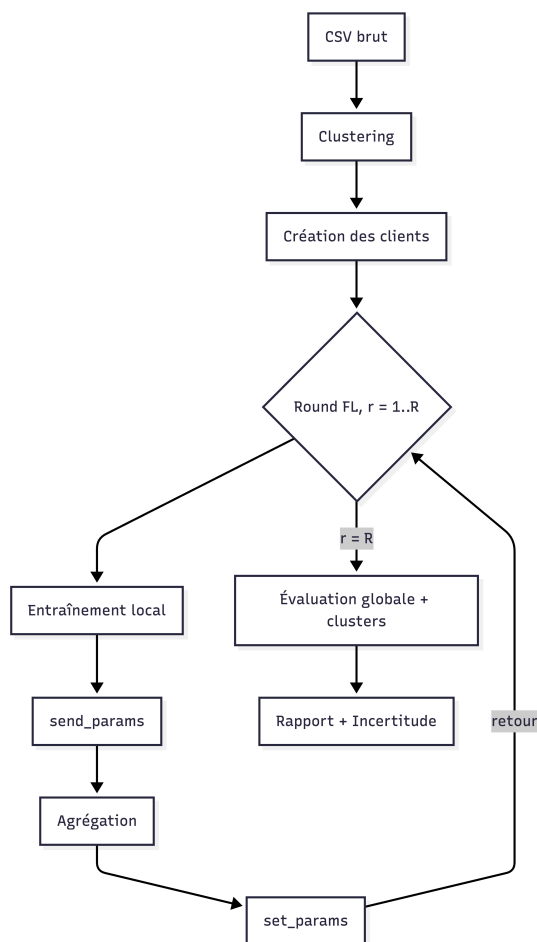


FIGURE 2 – Flux de données du pipeline GP-FL

## 2.3 Pourquoi les Processus Gaussiens ?

Trois raisons principales justifient ce choix.

D'abord, un GP ne donne pas seulement une prédiction : il donne aussi une mesure d'incertitude sur cette prédiction. C'est une propriété que la régression logistique ou les forêts aléatoires n'ont pas. En médecine, savoir qu'un modèle est incertain sur un cas est aussi important que la prédiction elle-même — cela permet de renvoyer ce patient vers un médecin.

Ensuite, les hyperparamètres du noyau GP ont une signification directe : une grande lengthscale pour une feature signifie que cette feature a peu d'impact dans ce cluster.

Enfin, le vecteur d'hyperparamètres est de petite taille ( $2 + d$  valeurs pour un noyau ARD), ce qui rend la communication fédérée très légère.

## 2.4 Stratégie de clustering

Deux méthodes sont utilisées selon la nature de la feature.

Si la feature a des seuils médicaux documentés (glycémie, BMI, HbA1c, etc.), le module `clustering.py` applique ces seuils directement — par exemple, pré-diabète si glycémie entre 100 et 126 mg/dL.

Sinon, K-means est utilisé avec une optimisation automatique du nombre de clusters  $k \in [2, 5]$  par score de silhouette.

Cette approche hybride a un avantage clair : les partitions médicales sont cliniquement interprétables. Mais elle a aussi une limite : certains clusters peuvent se retrouver avec une très forte majorité d’une seule classe (ex. cluster Normal pour HbA1c), ce qui rend la classification inutile dans ce groupe. Ce compromis entre interprétabilité clinique et faisabilité algorithmique est l’un des points les plus importants à analyser dans les résultats.

## 3 Conception Technique

### 3.1 Diagramme de classes UML

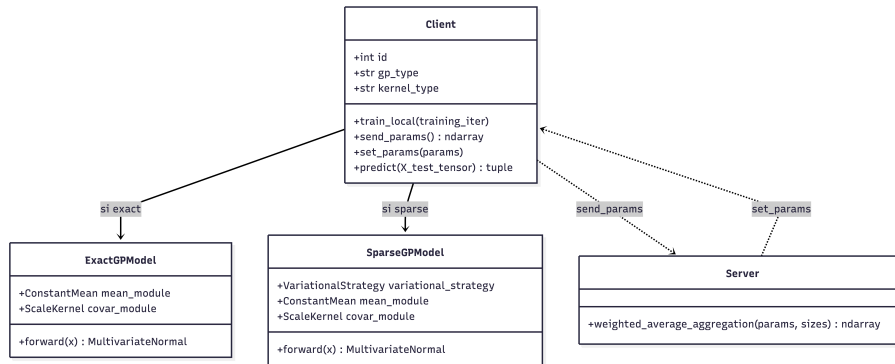


FIGURE 3 – Diagramme de classes UML du système GP-FL

### 3.2 Format de sérialisation des hyperparamètres

Les hyperparamètres échangés entre clients et serveur sont stockés dans un vecteur numpy de la forme suivante :

$$\theta = [\log(\text{output\_scale}), \log(\ell_1), \dots, \log(\ell_d), \text{mean\_constant}]$$

Les échelles (output scale et lengthscales) sont transmises en espace logarithmique. Cela a une raison concrète : faire une moyenne pondérée de valeurs en log-espace revient à calculer une moyenne géométrique en espace linéaire, ce qui est plus cohérent pour des grandeurs strictement positives. La constante de moyenne, elle, est transmise directement. À la réception, le client applique `exp()` aux composantes en espace logarithmique (output\_scale et lengthscales) pour récupérer les valeurs réelles positives, puis les réassigne aux tenseurs PyTorch correspondants. La constante de moyenne est quant à elle transmise directement en espace linéaire et ne nécessite pas de transformation inverse.

### 3.3 Agrégation fédérée

Le serveur calcule une moyenne pondérée par la taille d’entraînement de chaque client.

$$\theta_{\text{global}} = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} \cdot \theta_k$$

où  $n_k$  désigne le nombre d’exemples d’entraînement du client  $k$ , et  $K$  le nombre total de clients actifs dans le round courant. Cette pondération garantit que les clusters plus grands exercent une influence proportionnellement plus importante sur les hyperparamètres globaux. Un client avec 1 000 exemples pèse donc deux fois plus qu’un client avec 500 exemples. Cette approche est analogue au FedAvg de McMahan et al. (2017), adapté ici aux hyperparamètres GP plutôt qu’aux poids d’un réseau de neurones.

### 3.4 Gestion de la dimensionnalité

C’est un point technique important et non trivial. Quand on partitionne un dataset en clusters et qu’on applique `pd.get_dummies`, certaines catégories peuvent ne pas apparaître dans un cluster donné et le vecteur de features résultant est alors de taille différente d’un client à l’autre. Pour corriger ce problème, `expected_columns` est calculé une seule fois sur le dataset complet, avant la création des clients. Chaque client reçoit cette liste et remplit les colonnes manquantes avec des zéros. Cela garantit que tous les vecteurs `train_x` ont la même dimension `d`, ce qui est indispensable pour les noyaux ARD.

## 4 Analyse de Coûts

### 4.1 Ressources matérielles

Les expériences ont été conduites sur un GPU intégré AMD Radeon 780M (RDNA3, 12 compute units, TDP partagé avec le CPU). Ce GPU utilise une architecture UMA — sa mémoire est partagée avec la RAM système. Cela limite directement la taille des matrices de covariance qu’on peut stocker en GPU. C’est l’une des raisons pour lesquelles l’ExactGP n’est pas utilisable sur le Dataset 3 sans optimisation. La RAM système totale est estimée entre 16 et 32 Gb.

### 4.2 Ressources logicielles

| Bibliothèque | Version       | Rôle                        | Coût |
|--------------|---------------|-----------------------------|------|
| PyTorch      | 2.3.1+rocm5.7 | Tenseurs, autograd, GPU     | 0 €  |
| GPyTorch     | 1.15.1        | Modèles GP                  | 0 €  |
| scikit-learn | 1.8.0         | Métriques, K-means          | 0 €  |
| pandas       | 3.0.0         | Lecture et manipulation CSV | 0 €  |
| numpy        | 2.4.2         | Calculs numériques          | 0 €  |
| matplotlib   | 3.10.8        | Visualisations              | 0 €  |

TABLE 1 – Bibliothèques logicielles utilisées — coût total : 0 €

### 4.3 Temps de calcul

La complexité de l’ExactGP est  $O(n^3)$  par itération. En pratique, cela devient lent à partir de quelques milliers d’observations par cluster.

- Dataset 2 (768 lignes) : quelques secondes par feature.
- Dataset 1 ( $\sim 100\,000$  lignes,  $\sim 50\,000$  par cluster) : estimation 5–15 minutes par feature sur ce GPU, pour 3 rounds  $\times$  20 itérations.
- Dataset 3 ( $\sim 229\,000$  lignes, clusters de  $100\,000+$ ) : le mode ExactGP est impraticable. Le SparseGP avec 100 points inducteurs ramène le temps à environ 2–10 minutes par feature (estimation).

### 4.4 Complexité mémoire

| Mode     | Complexité mémoire                        | Utilisable si                    |
|----------|-------------------------------------------|----------------------------------|
| ExactGP  | $O(n^2)$ — matrice de covariance complète | $n < \sim 3\,000\text{--}5\,000$ |
| SparseGP | $O(nm)$ — $m$ = nb. points inducteurs     | $n$ arbitraire                   |

TABLE 2 – Complexité mémoire selon le mode GP

Pour  $n = 50\,000$  et des flottants 32 bits, la matrice ExactGP occuperait environ 10 Go, ce qui est incompatible avec un GPU intégré à mémoire partagée.

### 4.5 Taille des datasets

| Dataset   | Source            | Lignes          | Features | Cible                      |
|-----------|-------------------|-----------------|----------|----------------------------|
| Dataset 1 | Stroke prediction | $\sim 100\,000$ | 8        | diabetes (binaire)         |
| Dataset 2 | Pima Indians      | 768             | 8        | Outcome (binaire)          |
| Dataset 3 | BRFSS CDC Survey  | $\sim 229\,000$ | 8 / 22   | Diabetes__binary (binaire) |

TABLE 3 – Caractéristiques des trois datasets utilisés

### 4.6 Effort de développement estimé

| Composant                                    | Estimation  |
|----------------------------------------------|-------------|
| Client.py — GP exact/sparse, sérialisation   | $\sim 12$ h |
| clustering.py — seuils médicaux + K-means    | $\sim 8$ h  |
| evaluation.py — métriques + seuil F1 optimal | $\sim 4$ h  |
| uncertainty_evaluation.py                    | $\sim 6$ h  |
| visualize.py + comparison.py                 | $\sim 5$ h  |
| main.py — pipeline FL, gestion datasets      | $\sim 6$ h  |
| <b>Total</b>                                 | $\sim 41$ h |

TABLE 4 – Effort de développement estimé par composant

## 5 Premiers Résultats

Les expériences ont été menées avec 3 rounds fédérés et 20 itérations locales par round.

### 5.1 Dataset 1 — DPD (~100 000 patients)

Le modèle utilisé est un SparseGP avec noyau RBF et 100 points inducteurs. C’est le dataset où les performances sont les meilleures, ce qui est cohérent avec sa grande taille.

| Feature             | Cluster          | Accuracy | Precision | Recall | F1     |
|---------------------|------------------|----------|-----------|--------|--------|
| gender              | Female           | 0.9659   | 0.8384    | 0.6667 | 0.7427 |
|                     | Male             | 0.9528   | 0.8148    | 0.6687 | 0.7346 |
| age                 | Younger          | 0.9892   | 0.9583    | 0.3988 | 0.5633 |
|                     | Older            | 0.9333   | 0.8472    | 0.6727 | 0.7499 |
| hypertension        | No Hypertension  | 0.9666   | 0.8202    | 0.6486 | 0.7244 |
|                     | Hypertension     | 0.8864   | 0.8639    | 0.7019 | 0.7745 |
| heart_disease       | No Heart Disease | 0.9641   | 0.8341    | 0.6469 | 0.7286 |
|                     | Heart Disease    | 0.8669   | 0.8520    | 0.6872 | 0.7608 |
| smoking_history     | Never            | 0.9563   | 0.8029    | 0.6916 | 0.7431 |
|                     | Former           | 0.9336   | 0.8058    | 0.6977 | 0.7478 |
|                     | Current          | 0.9526   | 0.8552    | 0.6492 | 0.7381 |
|                     | No Info          | 0.9803   | 0.8564    | 0.5855 | 0.6955 |
| bmi                 | Underweight      | 0.9941   | 0.8000    | 0.3077 | 0.4444 |
|                     | Healthy          | 0.9800   | 0.8000    | 0.6228 | 0.7003 |
|                     | Overweight       | 0.9639   | 0.8263    | 0.6301 | 0.7150 |
|                     | Obesity I        | 0.9321   | 0.8598    | 0.6332 | 0.7293 |
|                     | Obesity II       | 0.9118   | 0.8333    | 0.7028 | 0.7625 |
|                     | Obesity III      | 0.8811   | 0.7342    | 0.7873 | 0.7598 |
| HbA1c_level         | Normal           | 1.0000   | 0.0000    | 0.0000 | 0.0000 |
|                     | Pre-diabetes     | 0.9369   | 0.6431    | 0.5221 | 0.5763 |
|                     | Diabetes         | 0.9454   | 0.9215    | 0.8396 | 0.8787 |
| blood_glucose_level | Normal           | 1.0000   | 0.0000    | 0.0000 | 0.0000 |
|                     | Pre-diabetes     | 1.0000   | 0.0000    | 0.0000 | 0.0000 |
|                     | Diabetes         | 0.9451   | 0.7963    | 0.7046 | 0.7477 |

TABLE 5 – Résultats par cluster — Dataset 1 (DPD)

Point notable : les clusters Normal et Pre-diabetes des features `HbA1c_level` et `blood_glucose_level` obtiennent  $F1 = 0.000$ . Dans ces groupes, presque tous les patients sont non-diabétiques — le modèle ne peut pas apprendre à détecter les cas positifs. En revanche, le cluster Diabetes de `HbA1c_level` atteint  $F1 = 0.879$  et celui de `blood_glucose_level`  $F1 = 0.748$ , ce qui confirme que le modèle performe correctement là où les deux classes sont représentées. C’est une limite inhérente au clustering par seuils médicaux : certains groupes sont quasi-purs et ne constituent pas un problème de classification équilibré.

## 5.2 Dataset 2 — Pima Indians (768 patients)

Le modèle utilisé est un ExactGP avec noyau RBF. Les performances sont plus modestes, ce qui est cohérent avec la petite taille du dataset.

| Feature          | Cluster               | Accuracy | Precision | Recall | F1     |
|------------------|-----------------------|----------|-----------|--------|--------|
| Glucose          | Normal                | 0.8108   | 0.0000    | 0.0000 | 0.0000 |
|                  | Pre-diabetes          | 0.7818   | 0.5000    | 0.7500 | 0.6000 |
|                  | Diabetes              | 0.7333   | 0.8125    | 0.7222 | 0.7647 |
| BloodPressure    | No Info               | 0.7143   | 0.6667    | 0.6667 | 0.6667 |
|                  | Low                   | 0.8208   | 0.6923    | 0.7941 | 0.7397 |
|                  | Normal                | 0.6552   | 0.6667    | 0.5714 | 0.6154 |
|                  | Elevated              | 0.6667   | 0.6000    | 1.0000 | 0.7500 |
| SkinThickness    | Cluster 0 (37.0–99.0) | 0.7778   | 0.7692    | 0.7692 | 0.7692 |
|                  | Cluster 1 (0.0–8.0)   | 0.7234   | 0.6190    | 0.7222 | 0.6667 |
|                  | Cluster 2 (10.0–24.0) | 1.0000   | 1.0000    | 1.0000 | 1.0000 |
|                  | Cluster 3 (25.0–36.0) | 0.6739   | 0.5417    | 0.7647 | 0.6341 |
| Insulin          | No Info               | 0.7200   | 0.6667    | 0.6000 | 0.6316 |
|                  | Normal                | 0.8077   | 0.6667    | 0.7500 | 0.7059 |
|                  | High                  | 0.7407   | 0.5882    | 1.0000 | 0.7407 |
| BMI              | Healthy               | 0.9048   | 0.6667    | 0.6667 | 0.6667 |
|                  | Overweight            | 0.7778   | 1.0000    | 0.2000 | 0.3333 |
|                  | Obesity I             | 0.6667   | 0.5333    | 0.9412 | 0.6809 |
|                  | Obesity II            | 0.8000   | 0.8000    | 0.6667 | 0.7273 |
|                  | Obesity III           | 0.7000   | 0.7143    | 0.8333 | 0.7692 |
| DiabetesPedigree | Cluster 0 (0.6–2.4)   | 0.6667   | 0.6296    | 0.8500 | 0.7234 |
|                  | Cluster 1 (0.1–0.6)   | 0.8087   | 0.6818    | 0.7895 | 0.7317 |
| Age              | Younger               | 0.8378   | 0.5263    | 0.7692 | 0.6250 |
|                  | Older                 | 0.7037   | 0.7234    | 0.7556 | 0.7391 |

TABLE 6 – Résultats par cluster — Dataset 2 (Pima Indians)

Point notable : le cluster Normal de **Glucose** obtient  $F1 = 0.000$ , ce qui reproduit le même problème de cluster quasi-pur observé dans le Dataset 1. Le cluster 2 de **SkinThickness** (10.0–24.0) atteint  $Accuracy = 1.000$  et  $F1 = 1.000$ , ce qui est suspect — ce cluster est probablement trop petit pour être représentatif. Enfin, le cluster Overweight de **BMI** présente une  $Precision = 1.000$  mais un  $Recall = 0.200$  et  $F1 = 0.333$ , ce qui indique que le modèle est très conservateur dans ce groupe et ne détecte qu’une fraction des cas diabétiques.

### 5.3 Dataset 3 — BRFSS (~229 000 patients)

Le modèle utilisé est un SparseGP avec noyau Matérn ARD et 100 points inducteurs. C’est le dataset le plus difficile, avec une prévalence du diabète d’environ 14%. Le Dataset 3 contient initialement 22 features. Une sélection a été effectuée par analyse de corrélation avec la variable cible, afin de ne retenir que les 8 features les plus pertinentes pour le modèle FL.

| Feature      | Cluster                    | Accuracy | Precision | Recall | F1     |
|--------------|----------------------------|----------|-----------|--------|--------|
| HighBP       | No High BP                 | 0.8597   | 0.2881    | 0.4119 | 0.3390 |
|              | High BP                    | 0.6294   | 0.4195    | 0.7493 | 0.5378 |
| HighChol     | Normal Cholesterol         | 0.8147   | 0.3095    | 0.5700 | 0.4012 |
|              | High Cholesterol           | 0.6653   | 0.4247    | 0.7228 | 0.5350 |
| BMI          | Underweight                | 0.8232   | 0.1569    | 0.4211 | 0.2286 |
|              | Healthy                    | 0.8534   | 0.2637    | 0.4380 | 0.3292 |
|              | Overweight                 | 0.7696   | 0.3422    | 0.5526 | 0.4227 |
|              | Obesity I                  | 0.6681   | 0.3877    | 0.7566 | 0.5127 |
|              | Obesity II                 | 0.6650   | 0.4757    | 0.7687 | 0.5877 |
|              | Obesity III                | 0.6793   | 0.5519    | 0.7961 | 0.6519 |
| HeartDisease | No Heart Disease           | 0.7630   | 0.3598    | 0.6536 | 0.4641 |
|              | Heart Disease              | 0.5626   | 0.4454    | 0.8227 | 0.5779 |
| GenHlth      | Excellent                  | 0.9123   | 0.2050    | 0.3520 | 0.2591 |
|              | Very Good                  | 0.8110   | 0.2478    | 0.4742 | 0.3255 |
|              | Good                       | 0.6561   | 0.3326    | 0.7078 | 0.4526 |
|              | Fair                       | 0.5987   | 0.4553    | 0.8012 | 0.5807 |
|              | Poor                       | 0.6490   | 0.5420    | 0.7770 | 0.6385 |
| PhysHlth     | Good (0–13 days)           | 0.7919   | 0.3857    | 0.5770 | 0.4623 |
|              | Poor (14+ days)            | 0.6788   | 0.4823    | 0.7170 | 0.5767 |
| DiffWalk     | No Difficulty Wal-<br>king | 0.8003   | 0.3561    | 0.5384 | 0.4287 |
|              | Difficulty Walking         | 0.6318   | 0.4777    | 0.7638 | 0.5878 |
| Age          | Younger                    | 0.8757   | 0.3826    | 0.4941 | 0.4312 |
|              | Older                      | 0.6880   | 0.4046    | 0.6853 | 0.5088 |

TABLE 7 – Résultats par cluster — Dataset 3 (BRFSS)

Point notable : on observe un pattern clair entre l’état de santé du cluster et la performance du modèle. Les clusters en bonne santé obtiennent des F1 très faibles (Excellent :  $F1 = 0.259$ , Underweight :  $F1 = 0.229$ ), car la prévalence du diabète y est très basse et les classes sont fortement déséquilibrées. En revanche, les clusters correspondant aux états dégradés obtiennent de bien meilleures performances : Obesity III atteint  $F1 = 0.652$ , GenHlth Poor  $F1 = 0.639$ , et Heart Disease  $F1 = 0.578$ . Ce résultat confirme que le clustering médical crée naturellement des groupes où le problème de classification est plus ou moins équilibré selon la prévalence réelle du diabète.

## 5.4 Incertitude GP — Dataset 1

| Confiance $\uparrow$ | Coverage (%) | Accuracy      | Precision     | F1            |
|----------------------|--------------|---------------|---------------|---------------|
| 0                    | 100.0        | 0.9549        | 0.9974        | 0.6273        |
| 10                   | 99.6         | 0.9554        | 0.9973        | 0.6247        |
| 20                   | 96.2         | 0.9588        | 1.0000        | 0.5390        |
| <b>30+</b>           | <b>37.9</b>  | <b>1.0000</b> | <b>0.0000</b> | <b>0.0000</b> |

TABLE 8 – Analyse de confiance GP — Dataset 1, feature HbA1c

| Catégorie de risque   | Échantillons (%) | Taux réel de diabète |
|-----------------------|------------------|----------------------|
| Low Risk (Confident)  | 48.6%            | 0.7%                 |
| Uncertain (Gray Zone) | 51.3%            | 15.3%                |
| High Risk (Confident) | 0.2%             | 100.0%               |

TABLE 9 – Catégories de risque GP — Dataset 1, feature HbA1c

Sur le Dataset 1, environ 50% des prédictions sont classées comme incertaines (écart-type médian  $\sigma \approx 0.17$ ). De plus,  $\sigma$  est légèrement plus élevé pour les prédictions incorrectes (0.1880) que pour les prédictions correctes (0.1805), ce qui montre que le modèle est mieux calibré sur ses erreurs. Pour la feature `blood_glucose_level`, 78.1% des prédictions sont marquées comme incertaines, ce qui reflète la difficulté de ce clustering. Cette capacité à quantifier l’incertitude est l’un des avantages principaux du GP par rapport à des modèles comme la régression logistique ou les forêts aléatoires.